E.C.H.O.

Earthquake & Conflict Hazard Observer

Distributed platform for real-time seismic monitoring and autonomous classification of natural events and human threats.

Fabiano Cacioli 2273170

Jacopo Rossi 2003424

Fabrizio Pietrobono 2272808

Emanuele Smisi 2278705

Luca Buonomini 2257472

1. The Geopolitical Context and the Objective

System Mission

E.C.H.O. is a platform for real-time seismic monitoring, designed to acquire high-frequency data from a distributed network of geographically dispersed sensors.

The system operates without interruption, ensuring continuous coverage even in scenarios of partial infrastructure failure.

Autonomous Event Classification

→ Natural Earthquakes

Detection of tectonic activity with characteristic frequencies in the 0.5–3.0 Hz range.

→ Conventional Explosions

Identification of detonations with spectral signatures in the 3.0–8.0 Hz range.

→ Nuclear Tests

Detection of ultra-high-energy events with a dominant frequency ≥ 8.0 Hz.

2. The Technical Challenges of the Project

A critical seismic monitoring system imposes stringent non-functional requirements that determine every architectural choice in the project.

Fault Tolerance

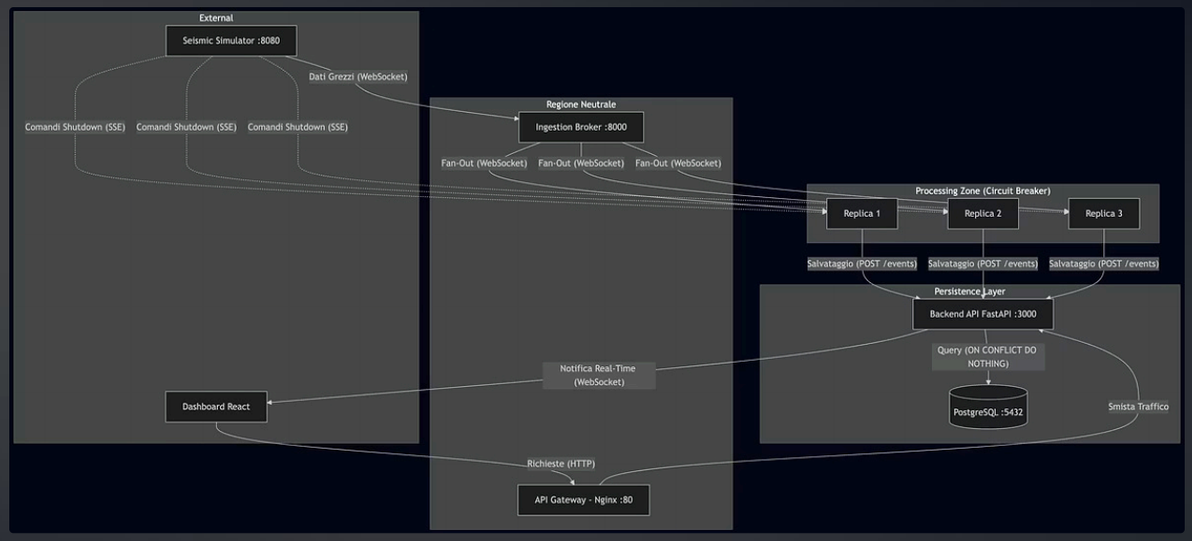
The system must never stop, even if a single node fails. Continuous availability is a non-negotiable requirement for a threat-detection infrastructure.

Real-Time Processing

Classification of each seismic event must occur within a few milliseconds of sample acquisition. Latency is unacceptable in early warning scenarios.

Idempotence

In a highly distributed system with parallel replicas, the same physical event must never be persisted more than once. Data uniqueness is mathematically guaranteed.



3. Microservices Architecture (Overview)

From Monolith to Microservices

The architecture completely abandons the monolithic paradigm in favor of containerized microservices with Docker. Each component has a single responsibility and clear boundaries.

Neutral Region

Lightweight, completely stateless routing. Responsible for directing traffic without analyzing the payload.

Processing Zone

Intensive DSP computation and classification. Independently scalable replicas from the routing layer.

Infrastructure as Code

The entire infrastructure is defined and deployed through a single `docker-compose.yml` file. This approach ensures:

- Absolute reproducibility of the environment on any host machine
- Versioning of the infrastructure together with the source code
- Deployment with a single command: `docker-compose up`
- Elimination of the "works on my machine" problem

4. The Principle of Neutrality — Gateway & Broker

The separation between routing and computation is the foundational architectural principle of E.C.H.O. The nodes in the Neutral Region operate as pure routers, never touching the data.

Stateless by Design

No node in the neutral region maintains state between one request and the next. Each request is handled in a completely independent way, eliminating session dependency.

Payload Blindness

Routing nodes never analyze the content of the message. They operate exclusively on headers, metadata, and routing rules, preserving the separation of responsibilities.

Independent Scalability

Because routing load is decoupled from computational load, neutral nodes can be scaled horizontally without affecting the Processing Replicas and vice versa.

5. The API Gateway — Security and Rate Limiting

Nginx as Single Entry Point

Nginx acts as the sole entry point (Reverse Proxy) to the entire network, exposing only port `80` to the outside world. All the complexity of the internal Docker network remains completely hidden from clients.

This pattern ensures that no internal service is ever directly reachable, drastically reducing the attack surface.

Rate Limiting with `limit_req_zone`

To protect the system from DoS attacks and misconfigured clients, Nginx implements the request limiting module:

Normal Threshold

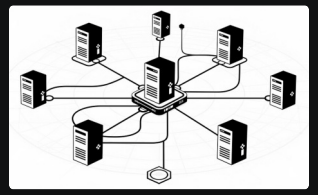
10 req/s per IP — standard operating limit for a human analyst or automated system.

Burst Buffer

Burst: 20 — absorbs legitimate transient spikes without rejecting valid requests.

6. The API Gateway — Load Balancing and Zero-Downtime

Nginx does not stop at security: it plays an active role in ensuring high availability through load balancing and passive health monitoring of backend replicas.

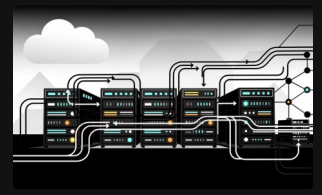


Distributed Load Balancing

Requests from Analysts are distributed evenly across all active backend replicas, preventing computational hotspots and maximizing overall throughput.

Passive Health Checks

Nginx continuously monitors API HTTP responses. If a Replica fails **3 times in a row**, it is automatically removed from the load balancing pool without human intervention.



Service Continuity

The combination of load balancing and health checks ensures an uninterrupted service for Analysts (zero-downtime), even during partial backend infrastructure failures.

7. Ingestion Broker — Discovery and Fan-Out

WebSocket Connectivity with Sensors

The Broker is the direct contact point with the seismic sensor network. It connects to each sensor via the WebSocket protocol, maintaining persistent bidirectional connections to receive high-frequency samples (20 Hz) efficiently.

Automatic Discovery

The Broker implements an auto-discovery mechanism that dynamically detects active sensors on the network. No manual configuration is required: new sensors are automatically integrated at the moment of connection.

Fan-Out Pattern to Replicas

Each raw reading received from a sensor is distributed in parallel to all 3 Processing Replicas simultaneously. This Fan-Out pattern ensures:

- Computational load balancing across replicas
- Redundancy in processing — no event is lost
- Decoupling data acquisition from data analysis
- Horizontal scalability of the processing layer without changes to the Broker

8. The Mathematical Core — The Window Problem

Fast Fourier Transform (FFT)

The Processing Replicas analyze the seismic signal by applying the Fast Fourier Transform to extract the frequency content and identify the dominant frequency of the event.

The analysis operates on time windows of only **5 seconds** — corresponding to **100 samples at 20 Hz** — to ensure near real-time classification.

Spectral Leakage: The Problem

Applying the FFT to a finite time window introduces a physical artifact known as **Spectral Leakage**: discontinuities at the window edges generate spurious spectral components that do not exist in the original signal.

In a critical classification context, this phenomenon leads to incorrect event identifications — an earthquake could be classified as an explosion, with severe operational consequences.

Spectral Leakage is the main enemy of accurate classification: it introduces "phantom" spectral energy that contaminates the FFT result.

9. DSP — Pink Noise and Thermal Drift

Before the window problem even comes into play, physical seismic sensors introduce two forms of disturbance that must be removed in signal preprocessing.

Pink Noise (1/f Noise)

Seismic sensors are affected by noise whose power spectral density decreases in proportion to the inverse of frequency (**1/f**). This creates very high spurious energy at low frequencies, which overlaps directly with the frequencies of seismic interest.

Thermal Drift

Slow temperature variations in the sensor installation environment induce a drift in the baseline signal — a quasi-DC component that appears as energy concentrated around zero Hz in the spectrum.

Insufficiency of Averaging

Simply removing the mean value (DC removal) is insufficient for such short windows. It does not eliminate the slow thermal trend or the 1/f energy content in sub-Hz frequencies, making a dedicated filtering approach necessary.

```python
def analyze_seismic_window(
    window,
    fs: int = 20,
    threshold: float = 40.0
) -> tuple[str, float, float, float]:
    window = np.asarray(window, dtype=float)

    # Detrend: rimuove offset DC
    window = window - np.mean(window)
    # Hanning window: previene spectral leakage
    window = window * np.hanning(len(window))

    freqs      = np.fft.rfftfreq(len(window), d=1 / fs)
    magnitudes = np.abs(np.fft.rfft(window))

    # Azzera DC e rumore a bassissima frequenza (< 0.2 Hz)
    magnitudes[freqs < 0.2] = 0.0

    max_idx       = int(np.argmax(magnitudes))
    dominant_freq = float(freqs[max_idx])
    max_magnitude = float(magnitudes[max_idx])

    if max_magnitude < threshold:
        return EVENT_TYPE_NONE, dominant_freq, max_magnitude, 0.0

    if 0.5 <= dominant_freq < 3.0:
        event_type = EVENT_TYPE_EARTHQUAKE
    elif 3.0 <= dominant_freq < 8.0:
        event_type = EVENT_TYPE_EXPLOSION
    elif dominant_freq >= 8.0:
        event_type = EVENT_TYPE_NUCLEAR
    else:
        event_type = "Unclassified Anomaly"

    severity = compute_severity_score(max_magnitude, threshold)
    return event_type, dominant_freq, max_magnitude, severity
```

10. The DSP Solution — Hanning Window and Filter

Hanning Window

Before applying the FFT, each time window is multiplied sample by sample by the Hanning function — a cosinoidal window that brings the values at the edges of the frame to zero.

This eliminates the discontinuities that caused Spectral Leakage, concentrating the spectral energy around the true dominant frequency of the signal and significantly increasing frequency resolution.

High-Pass Filter

After the window, a high-pass filter with cutoff frequency at **0.2 Hz** is applied. All spectral components below this threshold are brutally zeroed in the frequency domain.

This definitively removes:

- Pink Noise concentrated in sub-Hz frequencies
- Thermal drift and the residual DC component
- Any low-frequency artifact introduced by the sensors

The result is a clean spectrum from which it is possible to accurately extract the dominant frequency of the event.

11. Seismic Classification Rules

Once the dominant frequency (**f**) is extracted from the clean spectrum, event classification is performed using physically motivated frequency thresholds derived from the seismological literature.

🌍 Earthquake

0.5 Hz ≤ f < 3.0 Hz

Natural tectonic activity. Seismic waves generated by fault movements propagate at characteristically low frequencies, with main energy in the sub-3 Hz range.

💥 Conventional Explosion

3.0 Hz ≤ f < 8.0 Hz

Conventional detonations (chemical explosives, controlled demolitions). The propagation of the shock wave generates a spectral signature that is characteristically higher than earthquakes.

☢️ Nuclear-like Event

f ≥ 8.0 Hz

Underground nuclear tests or extremely high-energy events. The instantaneous release of energy generates very high-frequency components, clearly distinguishable from conventional explosions.

12. Severity Score — The Priority of the Alert

Beyond Binary Classification

Classifying the type of event is not enough to support operational decisions. Two earthquakes can both fall within the 0.5–3.0 Hz range, yet have radically different impacts on the population.

The system calculates a **Severity Score** between **0 and 100** to objectively quantify the relative danger of each detected event and define the analysts' response priority.

Calculation of the Severity Score

The score is based on the ratio between the maximum amplitude of the wave detected in the time window and the background noise threshold characteristic of the specific sensor:

Severity = f(A_max / Noise_Floor) × 100
Where A_max is the peak amplitude and Noise_Floor is the calibrated sensor noise baseline.

A high signal-to-noise ratio produces high scores, triggering priority alerts. Low scores indicate marginal events that can be monitored passively.

13. The Problem of Horizontal Scalability

The Fan-Out pattern, which ensures processing redundancy, introduces a critical data consistency problem: duplicate events in the database.

1

Parallel Reception

The Broker distributes the same seismic reading to all 3 Processing Replicas simultaneously via Fan-Out.

2

Independent Processing

All 3 Replicas analyze the same data window in parallel, producing identical classification results — correctly.

3

Triple Write

If all 3 perform a POST to the database, the same event is persisted 3 times. Dashboards become unusable, statistical counters are artificially inflated.

A mechanism for **absolute idempotency** was needed, one that did not depend on coordination between the replicas — which would have reintroduced coupling and latency.

14. Idempotency — UUIDv5 and Deterministic Generation

The Key to the Solution

Instead of coordinating replicas to avoid duplicate writes, E.C.H.O. adopts a radically different approach: each event's unique ID is **generated deterministically before saving**, using the same inputs available to all replicas.

UUIDv5: Hash-Based Generation

The UUIDv5 standard generates a 128-bit identifier by calculating the SHA-1 hash of a namespace and a name. The input for generation is the combination of:

- **Sensor_ID** — uniquely identifies the data source
- **Timestamp** — the exact moment the window was acquired
- **Event_Type** — the classification result

> Given the same triplet (Sensor_ID, Timestamp, Event_Type), **all replicas mathematically generate the same UUID** — without any communication between them.

```
INSERT_SQL = """
INSERT INTO seismic_events
    (event_id, sensor_id, event_type, dominant_hz, latitude, longitude, magnitude, severity_score, timestamp, replica_id)
VALUES
    ($1, $2, $3, $4, $5, $6, $7, $8, $9, $10)
ON CONFLICT (event_id) DO NOTHING;
"""
```

15. Idempotence — The PostgreSQL Database

Deterministic UUID generation would be incomplete without a persistence layer that honors it. PostgreSQL provides the native mechanism to handle conflicts silently and efficiently.

UNIQUE Constraint

The events table has a `UNIQUE` constraint on the UUID column. PostgreSQL rejects, at the schema level, any attempt to insert two rows with the same identifier.

ON CONFLICT DO NOTHING

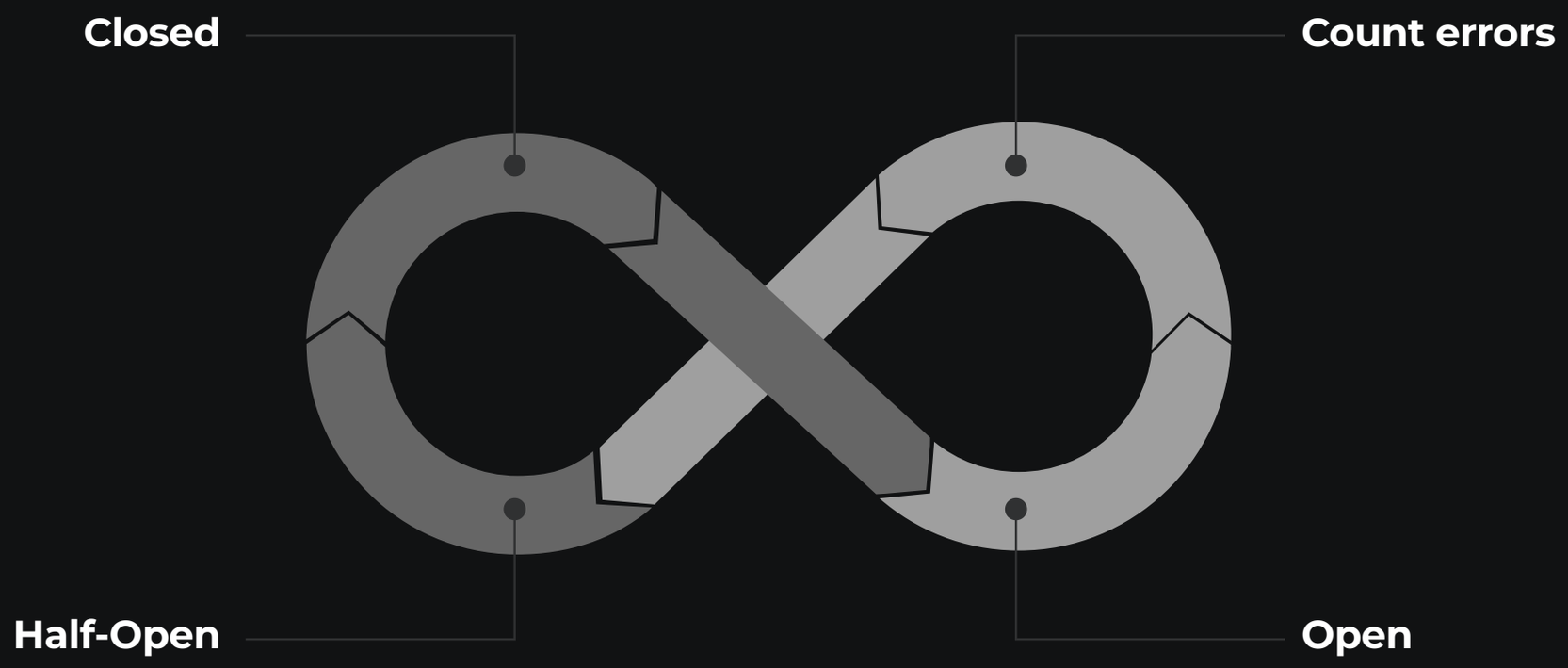
The backend uses the `INSERT ... ON CONFLICT DO NOTHING` query. If an UUID is already present, PostgreSQL silently ignores the operation without raising errors or exceptions.

First-Write Wins

The first replica to save the event wins the race condition. The other two receive a positive ACK (no error) and continue without any operational penalty.

16. Resilience — The Circuit Breaker Pattern

A distributed system must handle the failures of its workers without propagating cascading failure. The Circuit Breaker protects the Processing Replicas from a database that is temporarily unavailable.



State CLOSED — Normal Operation

The circuit is closed: all writes pass normally to PostgreSQL. Errors are counted internally. After **5 consecutive errors**, the circuit opens.

State OPEN — Protection Active

The circuit is open: the Replicas **temporarily discard** new events without attempting writes. This protects the node from accumulating timeouts that would exhaust resources.

17. Auto-Healing — Circuit Recovery

The Automatic Recovery Mechanism

After a timeout of **30 seconds** in the OPEN state, the Circuit Breaker automatically transitions to the **Half-Open** state. In this state, the Replica attempts a single test save — called a *Probe* — to the database.

Probe Successful → CLOSED

If the Probe is accepted by PostgreSQL, the circuit closes again and normal operation resumes automatically.

Probe Failed → OPEN

If the database is still unavailable, the circuit returns to the OPEN state and the 30-second timer restarts from the beginning.

Zero Human Intervention

The Circuit Breaker pattern with auto-healing allows the system to **fully self-heal** after a database failure, without requiring any intervention from operators.

This is essential for a critical monitoring system that must operate 24/7, especially in contexts where access to the infrastructure may be limited or delayed.

The combination of Passive Health Checks (Nginx), Circuit Breaker (Replica), and Idempotency (PostgreSQL) forms a multi-layer resilience triad.

18. Chaos Engineering — Fault Simulation

Testing Resilience in Production

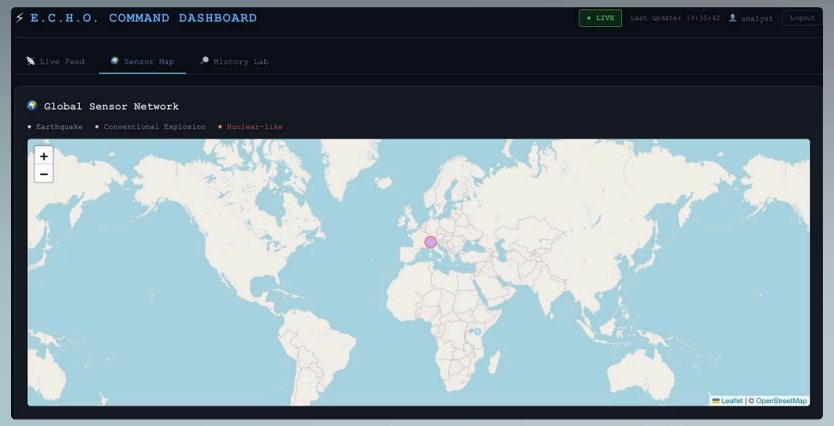
Fault tolerance mechanisms are meaningless if they are not tested systematically. E.C.H.O. implements an integrated Chaos Engineering module that injects real faults into the infrastructure during tests.

SSE Injection Mechanism

The Simulator communicates with the Replicas via **Server-Sent Events (SSE)**. This unidirectional protocol is ideal for distributing low-latency control commands without the overhead of bidirectional WebSocket.

The `SHUTDOWN` command instructs a Replica to intentionally terminate its own process, simulating a real crash. Following the SHUTDOWN, the system automatically verifies that:

- Nginx removes the Replica from the load-balancing pool
- The Broker reroutes Fan-Out to the remaining healthy nodes
- No seismic event is lost during failover
- The Analysts' dashboard continues receiving updates without interruption



19. The Intelligence Analysts' Dashboard

Single Page Application in React

The frontend is built as a React SPA, ensuring a smooth user experience without page reloads. Analysts access all operational data from a single interface.

Real-Time via WebSocket

The dashboard connects to the backend via a persistent WebSocket. New seismic and nuclear alerts arrive instantly without polling.

Interactive Maps

Each event is geolocated and displayed on interactive maps, enabling rapid geographic analysis of detected incidents.

Strategic Report Export

Analysts can export structured reports for strategic reporting and sharing with external operational units.